

# QSymGen2 Project

## User Documentation

**Revision Date: 2026.05.13**

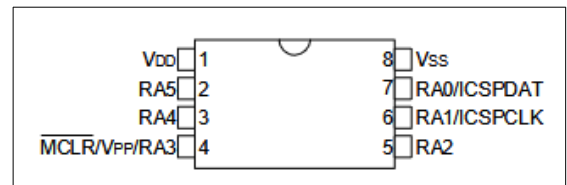
## Overview

QSymGen2 is a command line utility to simplify generating large pin-count symbols. It is intended for creating micro-controller symbols for use with the QMdbSim simulator project. It also serves as an example of reading/writing QSpice symbols and schematics using the QParser2 library. All of these projects are available in [my QSpice GitHub repository](#).

## Why This Tool Is Needed

We'll use a Microchip PIC16F15213/PIC16F15214 chip as an example ([data-sheet here](#)). This is a small 8-pin device with one input-only pin (RA3) and 5 “bi-directional pins.”

Because a QSpice DLL port can be only input or output, each bi-directional pin requires three DLL block ports and some additional circuitry. So, instead of 8 pins, this simple 8-pin device requires 18 DLL ports. This requires quite a bit of effort to create manually; imagine how onerous defining a 40-pin PIC18F4550 would be.



Rather than futz around with the QSpice GUI defining 18 pins, we can just create a text file to describe the 8 actual pins and let QSymGen2 generate the needed symbol and schematic files for us.

## Workflow

The workflow is fairly straight-forward:

1. Get the manufacturer's product data-sheet.
2. Define the pins in a text file (a “pin definitions file”).
3. Run `QSymGen2.exe` with the name of the pin definitions file.
4. Drop the generated symbol into your schematic and wire it up to whatever is needed for your project.
5. Open the generated schematic, right-click on the DLL symbol, and generate the DLL template code.
6. Add the included `GPIO_IMPL.qsch` file to your project.

Pretty simple.

## Create The Pin Definitions File

The distribution contains an example pin definitions file, `PIC16F1521x.qpins`. This is a simple ASCII text file. Note that the extension can be anything – I just used “.qpins” to help me recognize the file in File Explorer. The parser is pretty basic:

- Leading spaces are removed when parsing.
- Lines beginning with '\*' are comments and are ignored.
- Blank lines are ignored.

You can include a single line of text to include in the QSpice Symbol Properties “Description” item. For example:

```
D Microchip PIC16F15213/PIC16F15214
```

Will include “Microchip PIC16F15213/PIC16F15214” in the description.

To define pins:

- The first character determines the pin type.
- The next string defines the pin name used in the symbol and schematic.
- Additional text, if present, will be included for reference in the symbol. This is particularly useful to document additional pin peripheral functions, e.g., I<sup>2</sup>C, SPI, PWM, etc.

Pin types are:

- **i** = Input only pin.
- **o** = Output only pin.
- **b** = Bi-directional pin.
- **x** = A spacer element, i.e., an empty pin position in the symbol. (Currently, you must supply some pin name text for ‘X’ pins.)

Note: The example includes a “CLK” pin not present in the device. This is required for the QMdbSim project to advance the simulation. If not required for your project, omit it.

## A Note About Creating Pin Definition Files

Before moving on, a quick note about creating pin definition files....

You might be inclined to define the pins to match the pin order of the physical device. In fact, when entering the pins from the data-sheet, this is a sensible way to keep track and avoid errors. But there is no natural connection between physical device pin order and the schematic pin arrangement.

Once you’ve created the definition file, I suggest that you reorder the pins into logical groups. For example, I recommend placing all pins for a given micro-controller register together. This will cause them to be grouped together in the DLL uData array which will make mapping pins into a micro-controller register an array operation instead of using port names. (This support will be in future code-generation features.)

Finally, keep in mind that different part numbers may share the same pin functions but have different pin layouts. If they have the same functions and pin-count, there's no reason to create a new pin definition file. Just reuse the existing one.

## Generate Symbol & Schematic Files

**Caution:** *At the moment, QSymGen2 overwrites existing output files without warning.*

Run `QSymGen2.exe` from a command line to generate the symbol and schematic files. Example:

```
QSymGen2.exe .\Test\PIC16F1521x.qpins
```

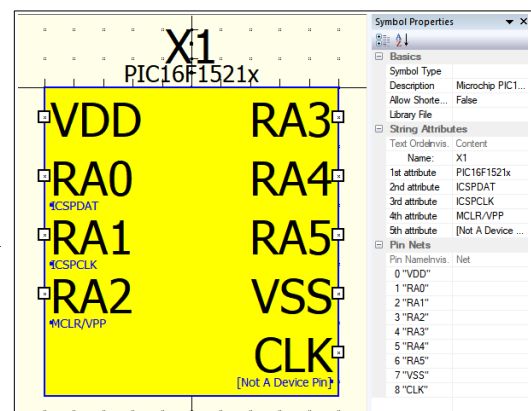
Runs QSymGen2 from the current folder against `PIC16F1521x.qpins` in the `Test` subfolder of the current folder. It will generate `PIC16F1521x.qsym` and `PIC16F1521x.qsch` in the `Test` subfolder.

## The Generated Symbol File

The generated symbol file for the example, `PIC16F1521x.qsym`, is a hierarchical schematic symbol. You can drag it onto a schematic and add the supporting circuitry. See `PIC16F15213_TopLevel.qsch` for an example that does nothing useful beyond demonstrating that the project will compile/run.

There are a couple of things to keep in mind:

- If a symbol pin is an input-only pin, you will need to wire it to something. Otherwise, QSpice will complain that the port is floating.
- When you drop a symbol onto a schematic, the symbol is copied into the schematic. Later changes to the symbol are not propagated into the schematic. If you change the symbol, delete it from the schematic and drag it back onto the schematic.



As mentioned, this is a hierarchical schematic symbol. It will attempt to load the schematic named in the 1<sup>st</sup> String Attribute, `PIC16F1521x.qsch` in this example. Naturally, you'll need to include that "child schematic" in your project.

## The Generated Schematic File

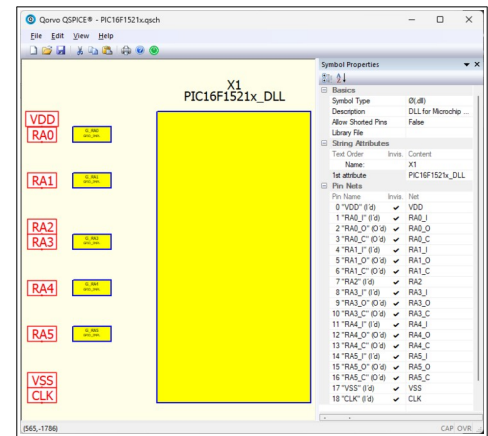
The generated schematic file contains the ports (which match the symbol pins), GPIO symbols for the bi-directional pins, and a DLL block. But it looks rather odd – there are no wires connecting anything and the DLL block doesn't even display any ports. What gives?

Well, I'm lazy. The various bits are "wired together" using net names. This, unfortunately, hides the pins/ports. Open the Symbol Properties and inspect the GPIO symbols and DLL block. You'll see the nets identified there.

Honestly, this isn't convenient for editing. On the other hand, it's not intended to be edited. The only reason I anticipate to edit the schematic is to add string attribute parameters to pass additional information to the DLL or change the default DLL block C++ file name (here, PIC16F1521x\_DLL).

Anyway, right-click on the DLL block to generate a C-Block template as usual.

Now, about those GPIO\_IMPL symbols. Those are hierarchical blocks that load the GPIO\_IMPL.qsch schematic. I have toyed around with different ways to implement the GPIO bi-directional functionality. I'm still thinking about the best way to handle this. Anyway, change the GPIO\_IMPL.qsch circuitry if you have a better solution. But, you will, of course, have to include a GPIO\_IMPL.qsch in your project.



## Future Plans

I will almost certainly make some changes. These might include:

- ~~Adding a warning before overwriting existing files.~~ Done.
- ~~Changing the schematic to use wires/nodes to allow DLL block pins to be displayed.~~ Nah.
- ~~Changing the X pin parsing to not require a dummy pin name.~~ Done.
- Allowing different pin data types, i.e., GPIO control pins could be a boolean. The clock pin could be a boolean.

More generally, I'll need to generate some template code for the DLL. For example:

- Pins that are grouped together internally by the micro-controller as a single register should be mapped to a single integer register value. In our example, the ports are RA0-RA5 which, internally is a single 8-bit port (only 6 bits used).
- Generating the all or part of the DLL template code directly instead of requiring the programmer to use the QSpice tools.

Let me know if you think of additional improvements.

## Conclusion

I hope that you find this tool to be useful. And, of course, let me know if you find problems or suggestions for improving the code or this documentation.

--robert